



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

CSE 4205: Digital Logic Design
Summer, 2024-2025

Aashnan Rahman, Jr. Lecturer, CSE

Lecture 5

Combinational Circuits I

Contents

1	Combinational Circuits	2
1.1	Definition	2
1.2	Characteristics	2
1.3	Combinational Circuit vs Sequential Circuit	2
1.4	Sequential or Combinational?	3
2	Combinational Circuits	4
2.1	Adders	4
2.1.1	Half Adder	5
2.1.2	Full Adder	6
2.1.3	Full Adder Using Half Adders:	7
2.2	Subtractors	9
2.2.1	Half Subtractor	9
2.2.2	Full Subtractor	10
2.2.3	Full Subtractor Using Half Subtractors:	13
2.3	Comparison between Adders and Subtractors	14
2.4	Different Forms of Adders	14
2.4.1	4-bit Ripple Carry Adder	14
2.4.2	Look-Ahead Carry Adder	16
2.4.3	Adder-Subtractor Circuit	17
2.5	Example	20
2.6	Limitations of the Adder-Subtractor Circuit	24
2.6.1	Some Circuit Implementations using Adders and Subtractors . . .	25

1 Combinational Circuits

Combination circuits are fundamental building blocks in Digital Logic Design (DLD) where the output depends only on the current input values and not on any previous states. Unlike sequential circuits, combination circuits do not contain memory elements such as flip-flops.

1.1 Definition

A combination circuit is a digital logic circuit in which the output is a pure function of the present input variables. It can be represented as:

$$\text{Output} = f(\text{Input variables}) \quad (1)$$

There is no feedback or memory, meaning past inputs do not affect the output.

1.2 Characteristics

- Output depends only on current inputs
- No memory elements (no flip-flops or latches)
- No clock signal required
- Faster operation compared to sequential circuits
- Easier to design and analyze

1.3 Combinational Circuit vs Sequential Circuit

Feature	Combinational Circuits	Sequential Circuits
Definition	Output depends only on present inputs	Output depends on present inputs and past states
Memory Element	No memory elements	Contains memory elements (flip-flops/latches)
Clock Signal	Not required	Required for synchronization
Feedback	No feedback path	Feedback is present
Speed	Faster	Comparatively slower
Design Complexity	Simpler to design	More complex design
Examples	Adders, Multiplexers, Encoders, Decoders	Counters, Registers, Shift registers

Table 1: Difference between Combinational and Sequential Circuits

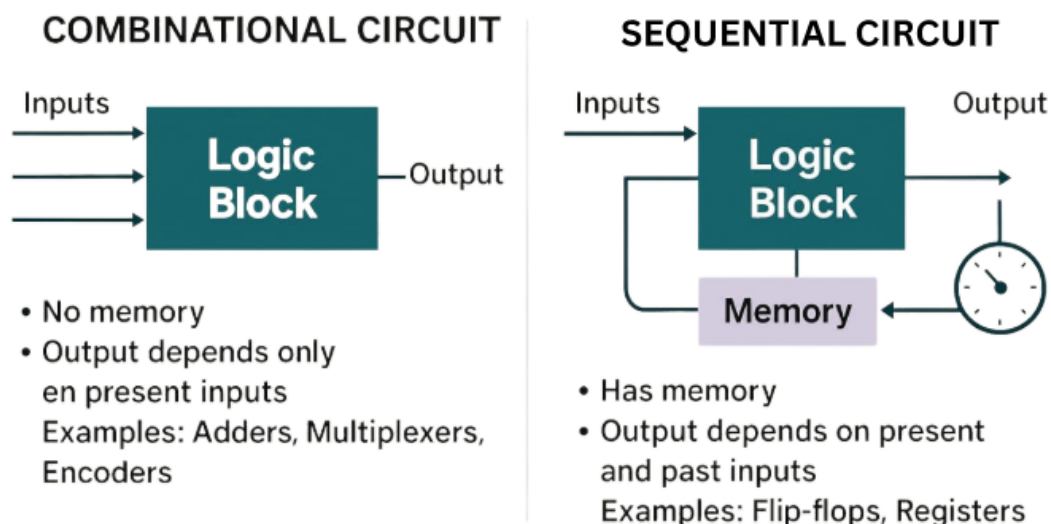


Figure 1: Combinational Circuit vs Sequential Circuit I

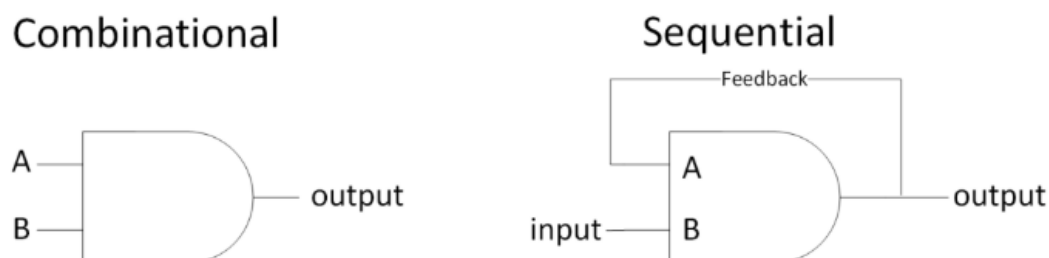


Figure 2: Combinational Circuit vs Sequential Circuit II

1.4 Sequential or Combinational?

Though the circuit [Figure 3](#) contains a feedback loop, it is still a combinational circuit because the output at any given time depends only on the current input values and not on any previous states. The feedback loop does not introduce memory elements or state retention, which is a key characteristic of sequential circuits.

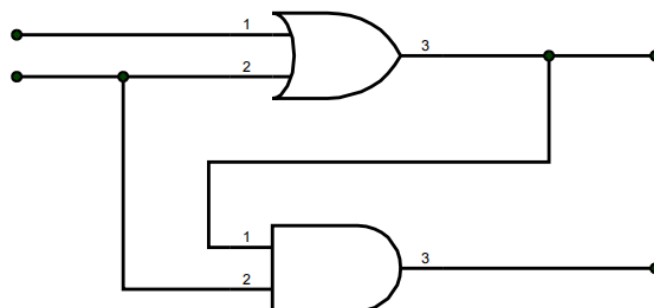


Figure 3: Combinational Circuit

A Sequential Circuit, on the other hand, would have memory elements such as flip-flops or latches that store the state of the circuit, and the output would depend on

both the current inputs and the stored state. In a sequential circuit, the feedback loop would be used to maintain or change the state based on the inputs and previous states. Like Figure 4, which contains a feedback loop that allows it to retain state information, making it a sequential circuit. Here the output at any given time depends not only on the current input values but also on the previous states of the circuit, which is a defining characteristic of sequential circuits.

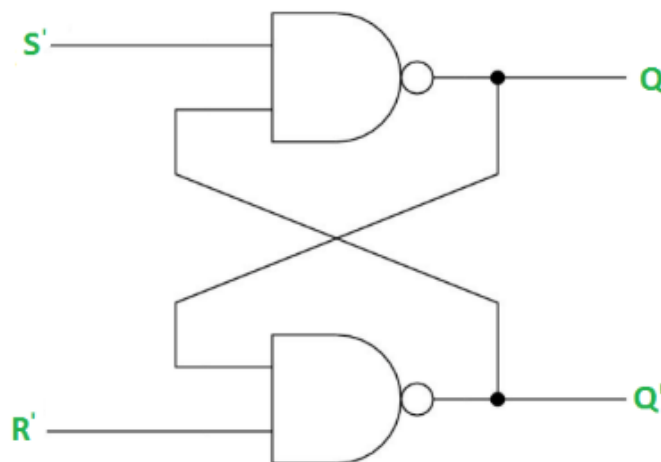


Figure 4: Sequential Circuit

2 Combinational Circuits

The combinational circuits that we will be covering in this lecture include:

- Adders
- Subtractors

2.1 Adders

Adders are fundamental combinational circuits used to perform binary addition. They are widely used in Arithmetic Logic Units (ALUs), digital processors, and computational systems. The two basic types of adders are the **Half Adder** and the **Full Adder**.

2.1.1 Half Adder

A Half Adder is a combinational circuit that adds two single-bit binary inputs and produces a Sum and a Carry output. It does not consider any carry input from previous stages.

Inputs: A, B

Outputs: Sum (S), Carry (C)

Truth Table:

A	B	S (Sum)	C (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 2: Truth Table of Half Adder

Boolean Equations(Using SoP):

$$S = A \cdot \bar{B} + \bar{A} \cdot B = A \oplus B \quad (2)$$

$$C = A \cdot B \quad (3)$$

Explanation: The sum is obtained using XOR operation, while the carry is generated using AND operation.

Circuit Implementation:

Using Basic Gates

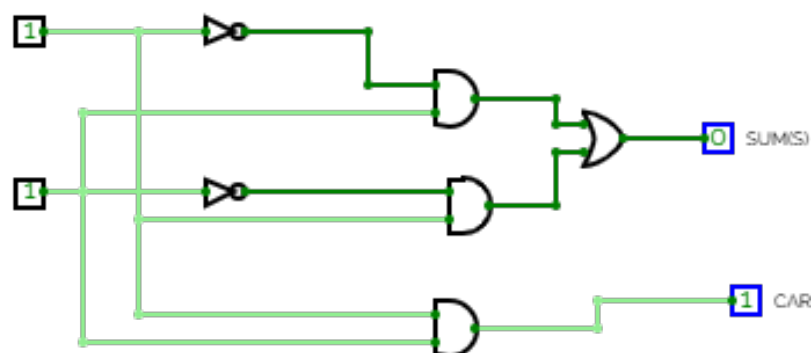


Figure 5: Half Adder Circuit(using Basic Gates)

Using XOR

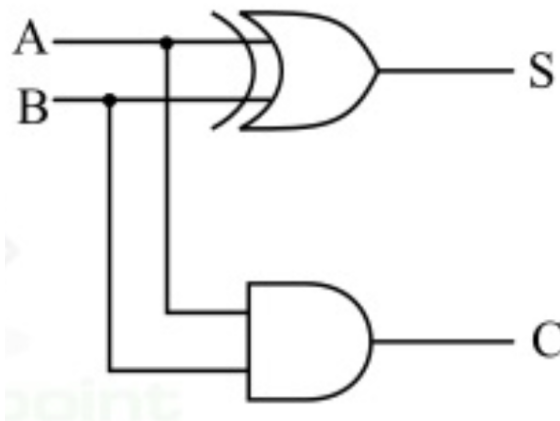


Figure 6: Half Adder Circuit

2.1.2 Full Adder

A Full Adder is a combinational circuit that adds three binary inputs: two significant bits and a carry-in from the previous stage. It produces a Sum and a Carry-out.

Inputs: A, B, C_{in}

Outputs: Sum (S), Carry-out (C_{out})

Truth Table:

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 3: Truth Table of Full Adder

Boolean Equations (Derivation from Truth Table):

Sum Output (S)

From the truth table, the Sum is 1 for minterms:

$$S(A, B, C_{in}) = \Sigma m(1, 2, 4, 7)$$

Expanding in SOP form:

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \quad (4)$$

Now simplifying (you can also simplify using K-map):

$$\begin{aligned} S &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \\ &= (\bar{A}\bar{B}C_{in} + ABC_{in}) + (\bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in}) \\ &= C_{in}(\bar{A}\bar{B} + AB) + \bar{C}_{in}(\bar{A}B + A\bar{B}) \\ &= C_{in}(AB + \bar{A}\bar{B}) + \bar{C}_{in}(A\bar{B} + \bar{A}B) \\ &= C_{in}(A \oplus B)' + \bar{C}_{in}(A \oplus B) \\ &= (A \oplus B)\bar{C}_{in} + (A \oplus B)'C_{in} \quad (\text{rearranged for clarity}) \\ &= (A \oplus B) \oplus C_{in} \\ &= A \oplus B \oplus C_{in} \end{aligned}$$

$$\boxed{S = A \oplus B \oplus C_{in}}$$

Carry Output (C_{out})

From the truth table, Carry is 1 for minterms:

$$C_{out}(A, B, C_{in}) = \Sigma m(3, 5, 6, 7)$$

Expanding in SOP form:

$$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \quad (5)$$

Now simplifying (you can also simplify using K-map):

$$\begin{aligned} C_{out} &= \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \\ &= \bar{A}BC_{in} + ABC_{in} + A\bar{B}C_{in} + ABC_{in} + AB\bar{C}_{in} + ABC_{in} \\ &\quad (\text{duplicated } ABC_{in} \text{ twice, since } X = X + X) \\ &= BC_{in}(\bar{A} + A) + AC_{in}(\bar{B} + B) + AB(\bar{C}_{in} + C_{in}) \\ &= BC_{in}(1) + AC_{in}(1) + AB(1) \\ &= AB + AC_{in} + BC_{in} \end{aligned}$$

$$\boxed{C_{out} = AB + BC_{in} + AC_{in}}$$

Explanation: The Full Adder extends the Half Adder by including a carry input, making it suitable for multi-bit binary addition.

Circuit Implementation:

2.1.3 Full Adder Using Half Adders:

A Full Adder can be constructed using two Half Adders and an OR gate. This is a commonly used implementation in digital systems to simplify the design of multi-bit adders.

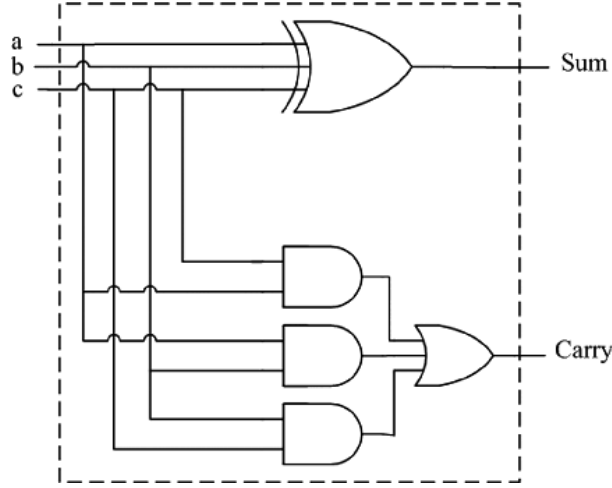


Figure 7: Full Adder Circuit

The first Half Adder adds the inputs A and B :

$$S_1 = A \oplus B \quad (6)$$

$$C_1 = A \cdot B \quad (7)$$

The Sum part of the Full Adder is obtained by adding the intermediate sum S_1 and the carry input C_{in} using the second Half Adder:

$$S = S_1 \oplus C_{in} \quad (8)$$

$$C_2 = S_1 \cdot C_{in} \quad (9)$$

The final carry output is obtained by OR-ing the two carry signals:

$$C_{out} = C_1 + C_2 \quad (10)$$

Justification Behind Carry Expression:

$$\begin{aligned} C_{out} &= \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \\ &= (\bar{A}BC_{in} + A\bar{B}C_{in}) + (AB\bar{C}_{in} + ABC_{in}) \\ &= C_{in}(\bar{A}B + A\bar{B}) + AB(\bar{C}_{in} + C_{in}) \\ &= C_{in}(A \oplus B) + AB(1) \\ &= AB + (A \oplus B)C_{in} \\ &= C_1 + (S_1)C_{in} \\ &= C_1 + C_2 \end{aligned}$$

Final Expressions:

$$S = A \oplus B \oplus C_{in} \quad (11)$$

$$C_{out} = AB + (A \oplus B)C_{in} \quad (12)$$

Thus, a Full Adder can be efficiently implemented using two Half Adders and one OR gate, reducing design complexity and making it suitable for modular circuit design.

Circuit Implementation:

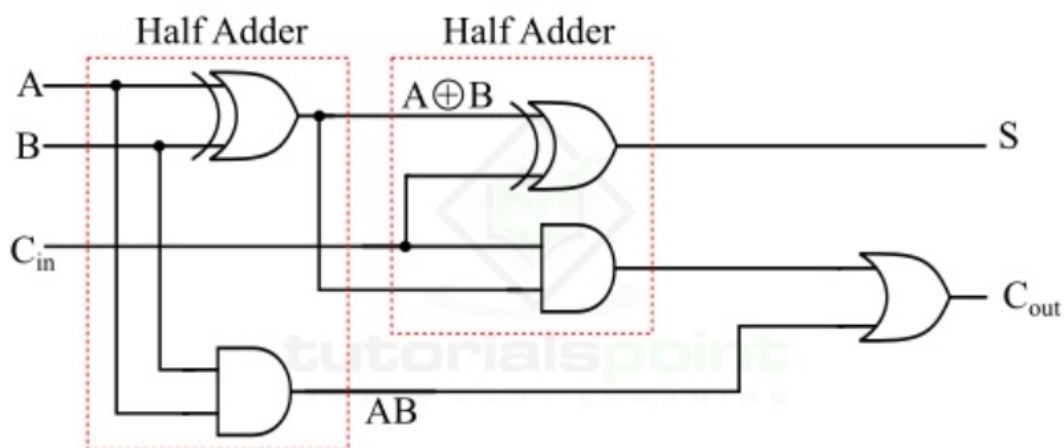


Figure 8: Full Adder Using Two Half Adders

2.2 Subtractors

Subtractors are combinational circuits used to perform binary subtraction. They are essential in digital systems for arithmetic operations. Similar to adders, subtractors are classified into **Half Subtractor** and **Full Subtractor**.

2.2.1 Half Subtractor

A Half Subtractor is a combinational circuit that subtracts two single-bit binary inputs and produces a Difference and a Borrow output. It does not consider any borrow input from previous stages.

Inputs: A, B

Outputs: Difference (D), Borrow (B_{out})

Truth Table:

A	B	D (Difference)	B_{out} (Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Table 4: Truth Table of Half Subtractor

How to construct the truth table:

- If $A > B$, then $B_{out} = 0$ (no borrow needed). Also, $D = 1$ since A can be greater than B only when $A = 1$ and $B = 0$. Hence,

$$D = A - B = 1 - 0 = 1$$

- If $A < B$, then $B_{out} = 1$ (borrow needed). Also, $D = 1$ since A is less than B only when $A = 0$ and $B = 1$. We borrow from the next higher bit (or an amount equal

to base of that number system, base 2), so A becomes $2 + 0$. Hence,

$$D = A - B = 2 - 1 = 1 \quad (\text{with borrow})$$

- If $A = B$, then $D = 0$ and $B_{out} = 0$ (no borrow needed).

Boolean Equations (Using SoP):

$$D = A \cdot \bar{B} + \bar{A} \cdot B = A \oplus B \quad (13)$$

$$B_{out} = \bar{A} \cdot B \quad (14)$$

Explanation: The difference is obtained using XOR operation (same as the sum in a half adder), while the borrow is generated only when $A = 0$ and $B = 1$ ($\bar{A}B$).

Circuit Implementation:

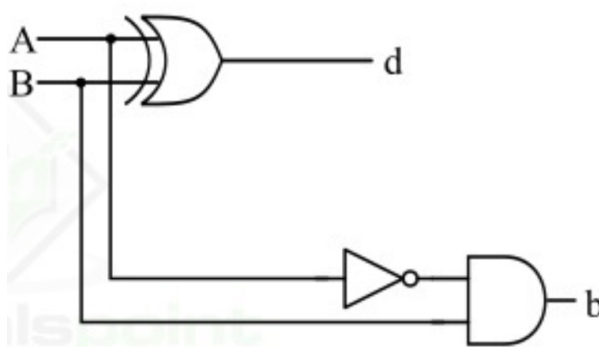


Figure 9: Half Subtractor Circuit

2.2.2 Full Subtractor

A Full Subtractor is a combinational circuit that subtracts three binary inputs: two significant bits and a borrow-in from the previous stage. It produces a Difference and a Borrow-out.

Inputs: A, B, B_{in}

Outputs: Difference (D), Borrow-out (B_{out})

Truth Table:

A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table 5: Truth Table of Full Subtractor

How to construct the truth table (Full Subtractor):

- The subtraction is always performed in the following way:

$$A - (B + B_{in})$$

- If $A > (B + B_{in})$, then $D = 1$ and $B_{out} = 0$ (no borrow needed).
Applicable when $A = 1$ and both B and B_{in} are 0.
- If $A < (B + B_{in})$, then $B_{out} = 1$ (borrow needed). In binary subtraction, we borrow from the next higher bit, which is equivalent to adding the base (2) to A .
Thus, A effectively becomes $A + 2$, and the difference is computed as:

$$D = (A + 2) - (B + B_{in})$$

All possible cases of D :

- Case 1: $A = 0$, $B + B_{in} = 1$ (Either $B = 1$ or $B_{in} = 1$)

$$D = (0 + 2) - 1 = 1$$

- Case 2: $A = 0$, $B + B_{in} = 2$ (Both $B = 1$ and $B_{in} = 1$)

$$D = (0 + 2) - 2 = 0$$

- Case 3: $A = 1$, $B + B_{in} = 2$ (Both $B = 1$ and $B_{in} = 1$)

$$D = (1 + 2) - 2 = 1$$

- If $A = (B + B_{in})$, then $D = 0$ and $B_{out} = 0$ (no borrow needed).

Boolean Equations (Derivation from Truth Table):

Difference Output (D)

From the truth table, the Difference is 1 for minterms:

$$D(A, B, B_{in}) = \Sigma m(1, 2, 4, 7)$$

Expanding in SOP form:

$$D = \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in} + ABB_{in} \quad (15)$$

Now simplifying (you can also simplify using K-map):

$$\begin{aligned} D &= \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in} + ABB_{in} \\ &= (\bar{A}\bar{B}B_{in} + ABB_{in}) + (\bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in}) \\ &= B_{in}(\bar{A}\bar{B} + AB) + \bar{B}_{in}(\bar{A}B + A\bar{B}) \\ &= B_{in}(AB + \bar{A}\bar{B}) + \bar{B}_{in}(A\bar{B} + \bar{A}B) \\ &= B_{in}(A \oplus B)' + \bar{B}_{in}(A \oplus B) \\ &= (A \oplus B)\bar{B}_{in} + (A \oplus B)'B_{in} \quad (\text{rearranged for clarity}) \\ &= (A \oplus B) \oplus B_{in} \\ &= A \oplus B \oplus B_{in} \end{aligned}$$

$$D = A \oplus B \oplus B_{in}$$

Borrow Output (B_{out})

From the truth table, Borrow is 1 for minterms:

$$B_{out}(A, B, B_{in}) = \Sigma m(1, 2, 3, 7)$$

Expanding in SOP form:

$$B_{out} = \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + \bar{A}BB_{in} + ABB_{in} \quad (16)$$

Now simplifying (you can also simplify using K-map):

$$\begin{aligned} B_{out} &= \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + \bar{A}BB_{in} + ABB_{in} \\ &= (\bar{A}B\bar{B}_{in} + \bar{A}BB_{in}) + (\bar{A}\bar{B}B_{in} + \bar{A}BB_{in}) + (\bar{A}BB_{in} + ABB_{in}) \\ &\quad (\text{duplicated } \bar{A}BB_{in} \text{ twice, since } X = X + X) \\ &= \bar{A}B(\bar{B}_{in} + B_{in}) + \bar{A}B_{in}(\bar{B} + B) + BB_{in}(\bar{A} + A) \\ &= \bar{A}B(1) + \bar{A}B_{in}(1) + BB_{in}(1) \\ &= \bar{A}B + \bar{A}B_{in} + BB_{in} \end{aligned}$$

$$B_{out} = \bar{A}B + \bar{A}B_{in} + BB_{in}$$

Explanation: The Full Subtractor extends the Half Subtractor by including a borrow input, making it suitable for multi-bit binary subtraction.

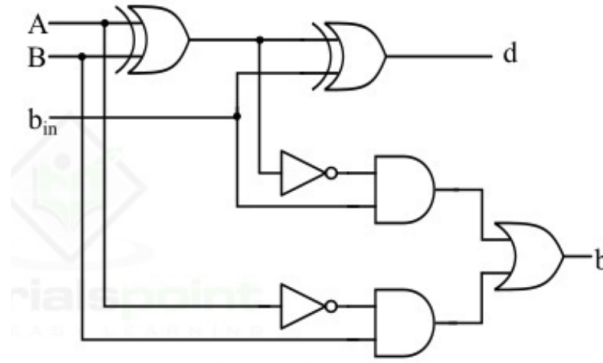


Figure 10: Full Subtractor Circuit

Circuit Implementation:**2.2.3 Full Subtractor Using Half Subtractors:**

A Full Subtractor can be constructed using two Half Subtractors and an OR gate. This modular approach simplifies the design and is commonly used in digital systems.

The first Half Subtractor subtracts B from A :

$$D_1 = A \oplus B \quad (17)$$

$$B_1 = \bar{A}B \quad (18)$$

The Difference part of the Full Subtractor is obtained by subtracting the borrow input B_{in} from the intermediate difference D_1 using the second Half Subtractor:

$$D = D_1 \oplus B_{in} \quad (19)$$

$$B_2 = \overline{D_1} \cdot B_{in} \quad (20)$$

The final borrow output is obtained by OR-ing the two borrow signals:

$$B_{out} = B_1 + B_2 \quad (21)$$

Justification Behind Borrow Expression:

$$\begin{aligned} B_{out} &= \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + \bar{A}BB_{in} + AB\bar{B}_{in} \\ &= (\bar{A}\bar{B}\bar{B}_{in} + \bar{A}B\bar{B}_{in}) + (\bar{A}\bar{B}B_{in} + AB\bar{B}_{in}) \\ &= \bar{A}B(\bar{B}_{in} + B_{in}) + B_{in}(\bar{A}\bar{B} + AB) \\ &= \bar{A}B(1) + B_{in}(A \oplus B)' \\ &= \bar{A}B + (A \oplus B)'B_{in} \\ &= B_1 + B_2 \end{aligned}$$

Final Expressions:

$$D = A \oplus B \oplus B_{in} \quad (22)$$

$$B_{out} = \bar{A}B + (A \oplus B)'B_{in} \quad (23)$$

Thus, a Full Subtractor can be efficiently implemented using two Half Subtractors and one OR gate, reducing design complexity and making it suitable for modular circuit design.

Circuit Implementation:

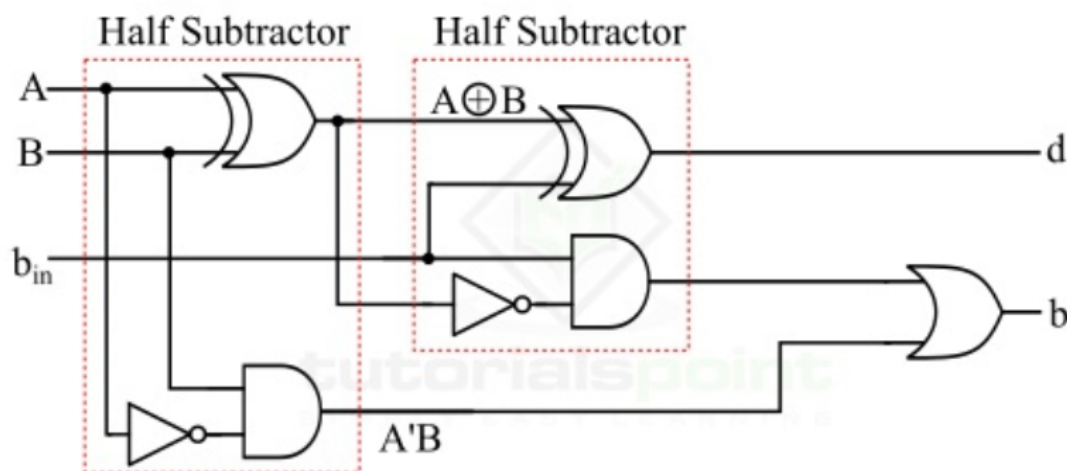


Figure 11: Full Subtractor Using Two Half Subtractors

2.3 Comparison between Adders and Subtractors

Feature	Adder	Subtractor
Definition	Performs binary addition of two or more bits	Performs binary subtraction of two or more bits
Basic Operation	Uses addition rules (0+0, 0+1, 1+1)	Uses subtraction rules with borrow concept
Carry/Borrow	Produces carry output	Produces borrow output
Main Components	Half Adder, Full Adder	Half Subtractor, Full Subtractor
Boolean Logic	Uses XOR and AND operations	Uses XOR, AND, and NOT operations
Complexity	Relatively simpler	Slightly more complex due to borrow handling
Applications	ALU, arithmetic operations, digital systems	ALU, subtraction operations, digital computation

Table 6: Comparison between Adders and Subtractors

2.4 Different Forms of Adders

2.4.1 4-bit Ripple Carry Adder

A 4-bit Ripple Carry Adder is a combinational circuit used to add two 4-bit binary numbers. It is constructed by connecting four Full Adders in series, where the carry output of each Full Adder is connected to the carry input of the next stage.

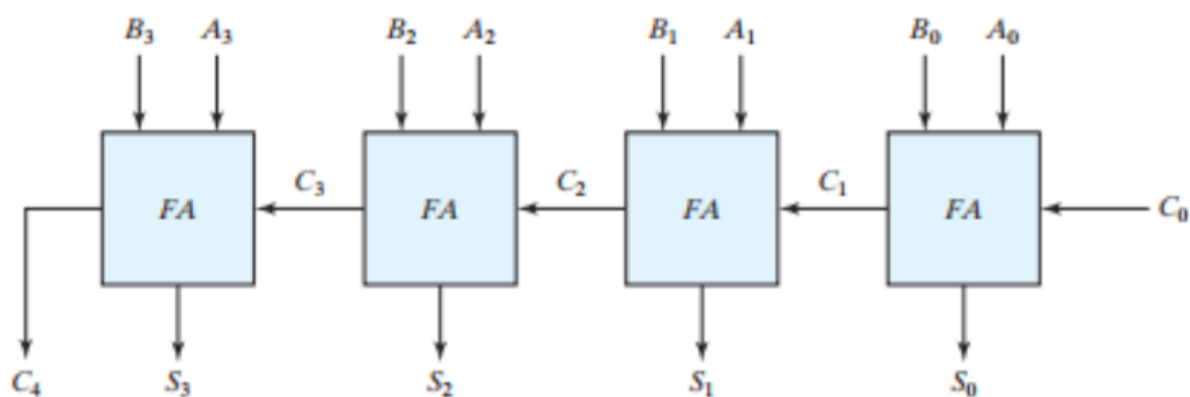


Figure 12: Ripple Carry Adder Circuit

Carry Propagation

In a Ripple Carry Adder, the carry generated at each stage is passed to the next Full Adder. This process is known as **carry propagation**. The carry must "ripple" through all intermediate stages before the final sum is obtained.

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4 \quad (24)$$

Thus, each Full Adder must wait for the carry input from the previous stage.

Characteristics

- Built using cascaded Full Adders
- Simple and easy to design
- No need for complex control logic
- Output depends on sequential carry propagation between stages

Delay and Performance Issue

The main disadvantage of the Ripple Carry Adder is its propagation delay. Since each Full Adder must wait for the carry from the previous stage, the total delay increases linearly with the number of bits.

The worst-case propagation delay is given by:

$$T_{total} = n \times T_{FA}$$

where n is the number of bits and T_{FA} is the delay of a single Full Adder.

This delay is illustrated in [Figure 12](#), where the carry must propagate through all stages before the final sum is produced.

Issue

- High propagation delay due to sequential carry dependence
- Not suitable for high-speed arithmetic operations

2.4.2 Look-Ahead Carry Adder

Due to the major limitation of the Ripple Carry Adder, which is the large propagation delay caused by sequential carry propagation, we look for a faster alternative known as the **Look-Ahead Carry Adder (LCA)** or **Carry Look-Ahead Adder (CLA)**.

In this approach, the carry is computed in advance using special signals called the **carry propagate** and **carry generate** terms.

Carry Propagate and Carry Generate:

For each bit position i :

$$P_i = A_i \oplus B_i \quad (\text{Carry Propagate}) \quad (25)$$

$$G_i = A_i \cdot B_i \quad (\text{Carry Generate}) \quad (26)$$

Carry Equations:

Instead of waiting for carry to ripple through each stage, the carries are computed directly using propagate (P_i) and generate (G_i) signals.

Let us consider the first bit (LSB), $i = 0$:

$$P_0 = A_0 \oplus B_0$$

$$G_0 = A_0 \cdot B_0$$

So,

$$S_0 = P_0 \oplus C_0$$

$$C_1 = G_0 + P_0 C_0$$

For the next bit, $i = 1$:

$$P_1 = A_1 \oplus B_1$$

$$G_1 = A_1 \cdot B_1$$

So,

$$S_1 = P_1 \oplus C_1$$

Now substituting C_1 :

$$\begin{aligned} C_2 &= G_1 + P_1 C_1 \\ &= G_1 + P_1 (G_0 + P_0 C_0) \\ &= G_1 + P_1 G_0 + P_1 P_0 C_0 \end{aligned}$$

For $i = 2$:

$$P_2 = A_2 \oplus B_2$$

$$G_2 = A_2 \cdot B_2$$

So,

$$S_2 = P_2 \oplus C_2$$

Substituting C_2 :

$$\begin{aligned} C_3 &= G_2 + P_2 C_2 \\ &= G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0) \\ &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \end{aligned}$$

$$C_1 = G_0 + P_0 C_0 \quad (27)$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 \quad (28)$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \quad (29)$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \quad (30)$$

Key Idea: All carry signals are generated in parallel using combinational logic, which significantly reduces the overall propagation delay compared to the Ripple Carry Adder.

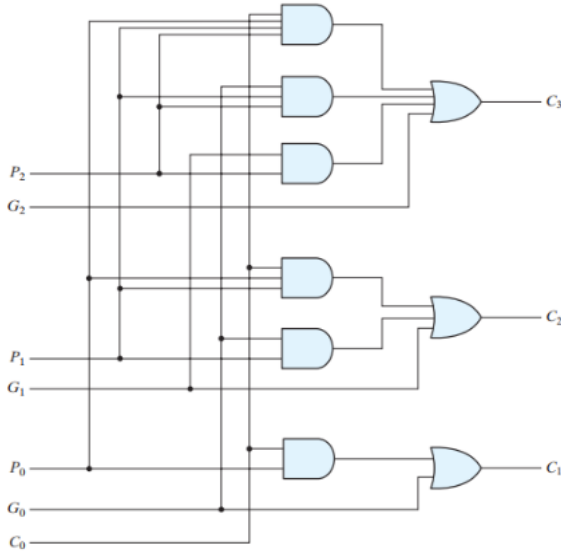


Figure 13: Propagator and Carry Generator

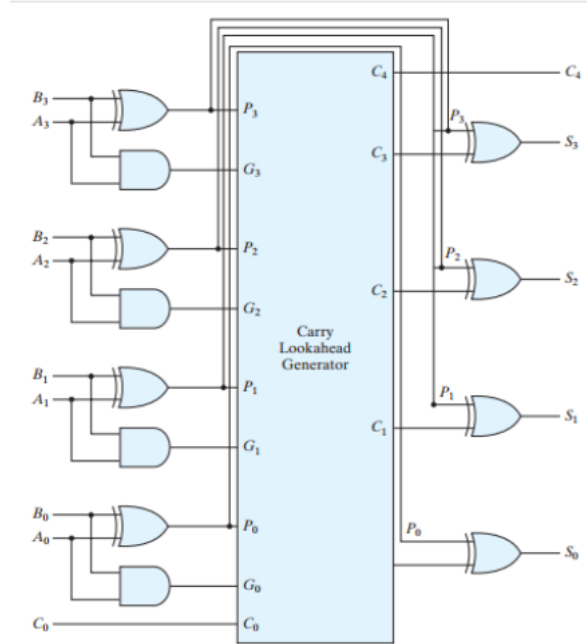
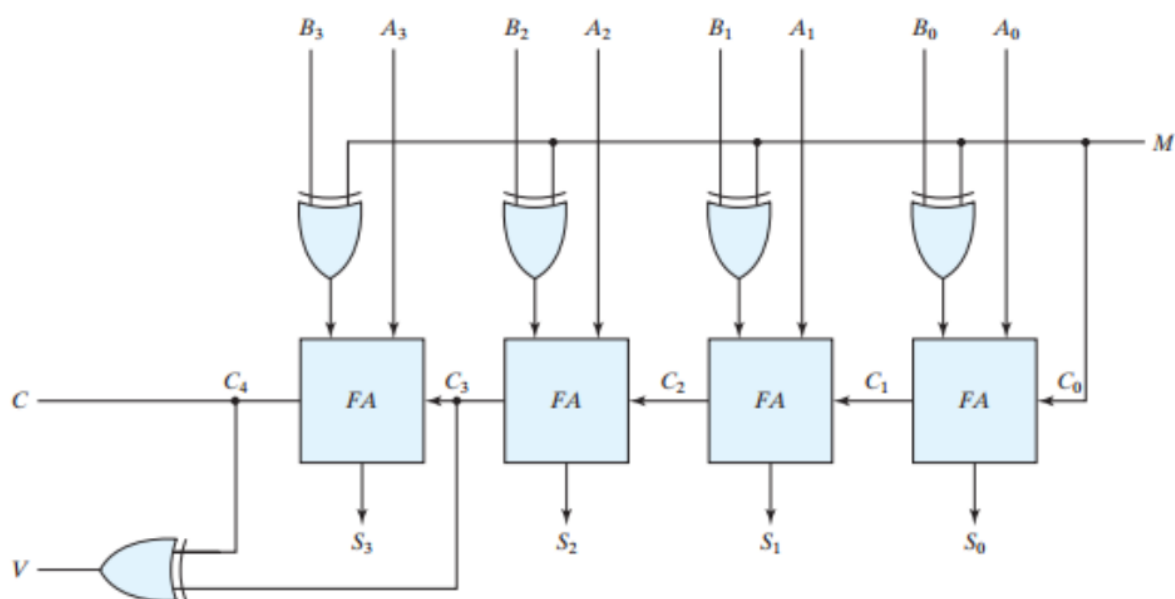


Figure 14: Look-Ahead Carry Adder Circuit

2.4.3 Adder-Subtractor Circuit

An Adder-Subtractor is a versatile combinational circuit that can perform both binary addition and binary subtraction using the same hardware. Instead of building separate circuits for addition and subtraction, this unified design minimizes area and power consumption by reusing the existing Full Adder structure with minimal additional logic. It is widely used in Arithmetic Logic Units (ALUs) and microprocessors.

Figure 15: n -bit Adder-Subtractor Circuit**Circuit Implementation:****Components:**

- **n Full Adders (FA):** Connected in a ripple-carry configuration to perform the core arithmetic.
- **n XOR Gates:** One for each bit of the B input, controlled by a single Mode signal.
- **Mode Input (M):** A control signal that selects the operation ($M = 0$ for Addition, $M = 1$ for Subtraction).
- **Inputs:** $A_{n-1} \dots A_0$, $B_{n-1} \dots B_0$, and Mode M .
- **Outputs:** Result $S_{n-1} \dots S_0$, Final Carry-out (C_{out}), and Overflow (V).

Operation Principle:

The key idea behind this circuit is the property of the XOR gate. When one input of an XOR gate is held at logic 0, the other input passes through unchanged. When it is held at logic 1, the other input is inverted.

$$X_i = B_i \oplus M \quad (31)$$

Where X_i is the modified version of B_i fed into the Full Adder. Additionally, the Mode signal M is directly connected as the initial carry-in (C_0) to the first Full Adder.

Operation Table:

Mode (M)	Input to FA (X_i)	Initial Carry ($C_0 = M$)	Operation
0	B_i	0	Addition ($A + B$)
1	$\bar{B}_i$	1	Subtraction ($A - B$)

Table 7: Adder-Subtractor Modes of Operation

Table 8: When $M = 0$ (Addition)

$$\begin{array}{rclcl}
 B_i & \oplus & M & = & X_i \\
 0 & \oplus & 0 & = & 0 \\
 1 & \oplus & 0 & = & 1 \\
 X_i & = & B_i & \text{(unchanged)}
 \end{array}$$

Table 9: When $M = 1$ (Subtraction)

$$\begin{array}{rclcl}
 B_i & \oplus & M & = & X_i \\
 0 & \oplus & 1 & = & 1 \\
 1 & \oplus & 1 & = & 0 \\
 X_i & = & \bar{B}_i & \text{(1's complement or bit flipped)}
 \end{array}$$

How Addition Works ($M = 0$)

When the Mode input is set to 0:

- The XOR gates act as buffers: $X_i = B_i \oplus 0 = B_i$. The original B input is passed to the Full Adders without modification.
- The initial carry-in is $C_0 = 0$.
- The circuit simply performs binary addition: $S = A + B$.

How Subtraction Works ($M = 1$) — Using Two's Complement

When the Mode input is set to 1:

- The XOR gates act as inverters: $X_i = B_i \oplus 1 = \bar{B}_i$. Every bit of B is complemented.
- The initial carry-in is $C_0 = 1$.
- The circuit computes: $S = A + \bar{B} + 1$.

By the definition of two's complement, the two's complement of a binary number B is given by $\bar{B} + 1$. Therefore, adding A to the two's complement of B effectively performs subtraction:

$$S = A + (\bar{B} + 1) = A - B$$

Boolean Equations for the i -th Stage:

For the i -th Full Adder stage ($i = 0$ to $n - 1$), the intermediate input and the standard Full Adder equations are:

$$X_i = B_i \oplus M \quad (32)$$

$$C_{i+1} = A_i X_i + X_i C_i + A_i C_i \quad \text{(Carry-out of stage } i) \quad (33)$$

$$S_i = A_i \oplus X_i \oplus C_i \quad \text{(Sum/Difference output)} \quad (34)$$

With the boundary conditions:

$$C_0 = M \quad (35)$$

$$C_{out} = C_n \quad (\text{Final carry-out of the MSB}) \quad (36)$$

Overflow Detection (V)

Overflow occurs when the result of an arithmetic operation exceeds the range that can be represented with the given number of bits. The detection of overflow depends on whether the numbers are treated as signed or unsigned:

- **For Unsigned Numbers:** Overflow is indicated by the final carry-out, $C_{out} = C_n$. If $C_n = 1$, the result is too large to fit in n bits.
- **For Signed Numbers (Two's Complement):** Overflow occurs when the carry into the most significant bit (MSB) is different from the carry out of the MSB. This is because a change in the sign bit indicates an erroneous result.

$$V = C_{n-1} \oplus C_n$$

Where C_{n-1} is the carry generated by the $(n-1)$ -th Full Adder (into the MSB), and C_n is the final carry-out of the MSB.

Conditions for Overflow in Signed Addition/Subtraction:

- Adding two positive numbers yields a negative result ($V = 1$).
- Adding two negative numbers yields a positive result ($V = 1$).
- Subtracting a negative from a positive (or vice versa) does not cause overflow.

Thus, the combined circuit provides both the arithmetic result S and the necessary status flags (C_{out} for unsigned overflow and V for signed overflow).

2.5 Example

Let us consider a 4-bit Adder-Subtractor with:

$$A = 0110 \quad (6)$$

$$B = 0010 \quad (2)$$

Addition ($M = 0$)

Since $M = 0$,

$$X = B \oplus M = B = 0010$$

$$C_0 = 0$$

Final carry:

$$C_4 = 0$$

Table 10: Addition: $6 + 2$

Bit i	A_i	B_i	M	X_i	C_i	S_i
0	0	0	0	0	0	0
1	1	1	0	1	0	0
2	1	0	0	0	1	0
3	0	0	0	0	1	1

Therefore,

$$S = 1000_2 = 8_{10}$$

$$0110 + 0010 = 1000$$

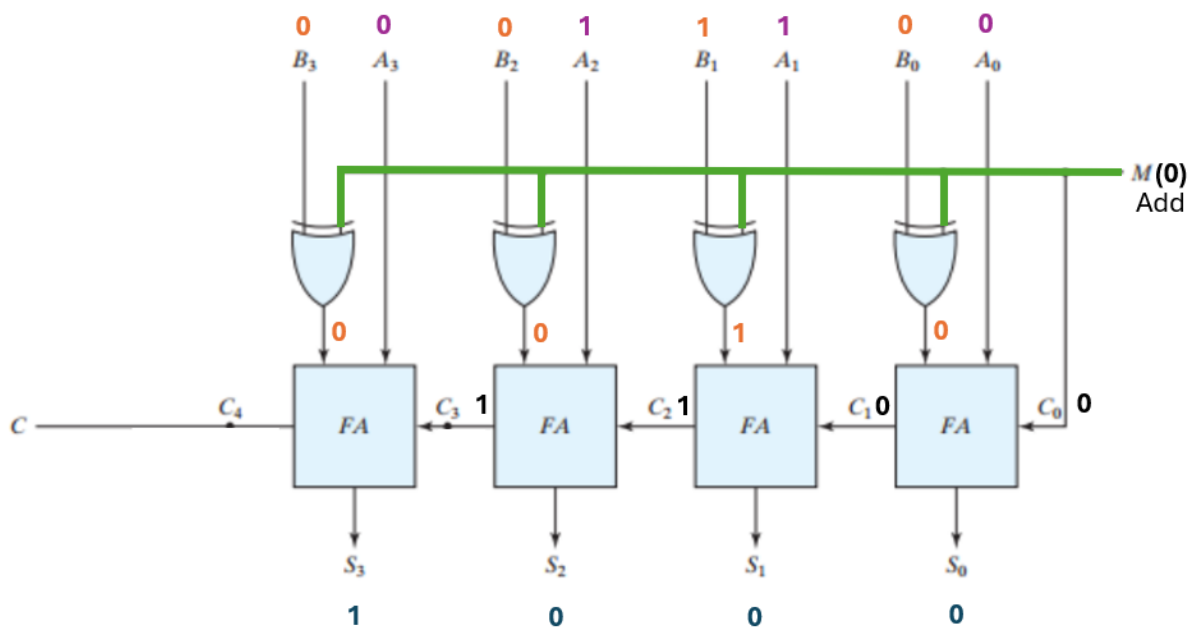


Figure 16: Addition using Adder-Subtractor Circuit

Subtraction ($M = 1$)

Since $M = 1$,

$$X = B \oplus M = \overline{B} = 1101$$

$$C_0 = 1$$

Final carry:

$$C_4 = 1$$

Therefore,

$$S = 0100_2 = 4_{10}$$

Table 11: Subtraction: $6 - 2$

Bit i	A_i	B_i	M	X_i	C_i	S_i
0	0	0	1	1	1	0
1	1	1	1	0	1	0
2	1	0	1	1	1	1
3	0	0	1	1	1	0

$$0110 + 1101 + 1 = 0100$$

Hence,

$$6 - 2 = 4$$

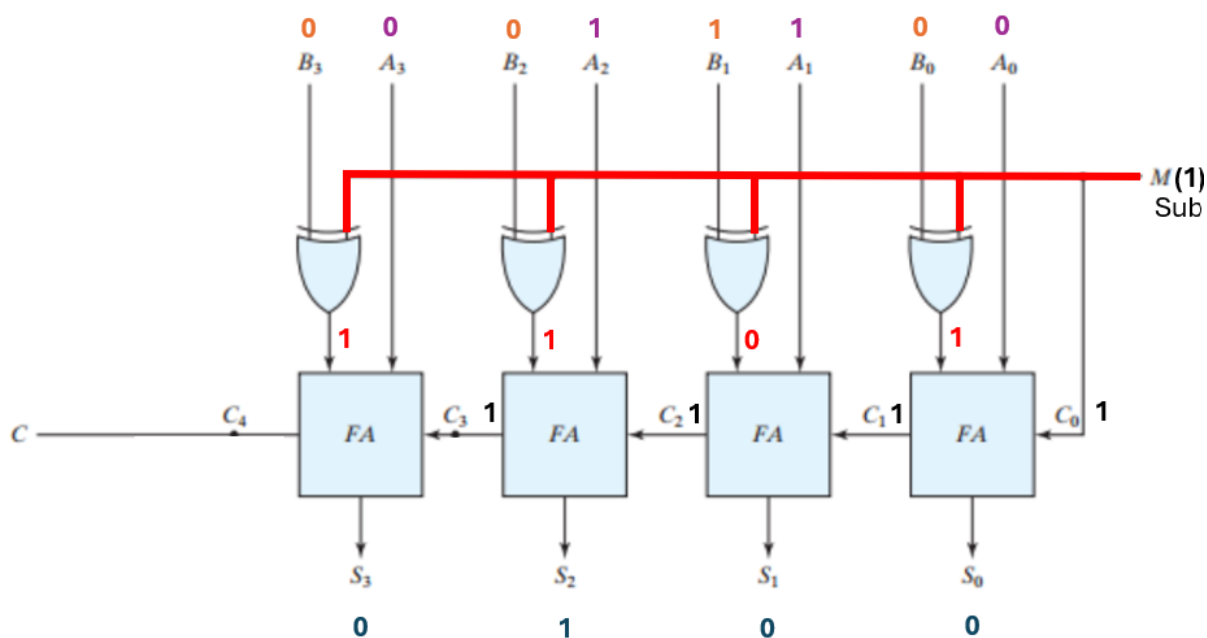


Figure 17: Subtraction using Adder-Subtractor Circuit

Subtraction Example: $2 - 6$

Let

$$A = 0010 \quad (2)$$

$$B = 0110 \quad (6)$$

$$M = 1 \quad (\text{Subtraction})$$

Since $M = 1$,

$$X = B \oplus M = \overline{B} = 1001$$

$$C_0 = 1$$

The Full Adders therefore perform:

$$A + X + C_0 = 0010 + 1001 + 0001$$

Table 12: Subtraction: $2 - 6$

Bit i	A_i	B_i	M	X_i	C_i	S_i
0	0	0	1	1	1	0
1	1	1	1	0	1	0
2	0	1	1	0	1	1
3	0	0	1	1	0	1

Thus,

$$S = 1100$$

and

$$C_4 = 0.$$

Since the most significant bit of the result is 1, the output represents a negative number in two's complement form.

To determine its decimal value:

$$\begin{aligned} 1100 &\xrightarrow{\text{invert}} 0011 \\ &\xrightarrow{+1} 0100 \end{aligned}$$

Therefore,

$$1100_2 = -4_{10}$$

which verifies that

$$2 - 6 = -4.$$

This example demonstrates that the Adder-Subtractor circuit naturally produces negative results in two's complement representation when the minuend is smaller than the subtrahend.

You can try out different combinations of A , B , and M to observe how the circuit behaves for various addition and subtraction scenarios, including cases that result in overflow or negative outputs. You will find a simplified circuit [here](#).

2.6 Limitations of the Adder-Subtractor Circuit

- **Propagation Delay**

The circuit is usually built using Ripple Carry Adders. During both addition and subtraction, the carry/borrow must propagate through multiple Full Adders, resulting in significant delay for large bit-widths.

- **Overflow Issues**

When operating on **signed numbers**, overflow may occur if the result exceeds the representable range. Additional overflow detection logic is required.

- **Scalability Challenges**

As the number of bits increases, the carry propagation delay increases linearly, making the circuit unsuitable for high-speed arithmetic applications.

Final Summary:

$$\boxed{S = A + B \quad (\text{when } M = 0)} \quad (37)$$

$$\boxed{S = A - B \quad (\text{when } M = 1)} \quad (38)$$

$$\boxed{V = C_{n-1} \oplus C_n \quad (\text{Signed Overflow})} \quad (39)$$

$$\boxed{C_{out} = C_n \quad (\text{Unsigned Overflow})} \quad (40)$$

By integrating XOR gates with a ripple-carry adder, the Adder-Subtractor circuit provides a flexible, efficient, and widely used building block for arithmetic processing in digital systems.

2.6.1 Some Circuit Implementations using Adders and Subtractors

- **Magnitude Comparator using a Subtractor**

Design a circuit that takes two n -bit numbers, A and B , as inputs and determines whether $A > B$, $A = B$, or $A < B$. Use a subtractor to compute $A - B$ and determine the result from the sign bit and zero detection logic.

- **Absolute Difference Circuit**

Design a circuit that computes the absolute difference between two numbers:

$$|A - B|$$

If $A < B$, the result should be converted to its positive equivalent before being displayed.

- **Adder/Subtractor Unit**

Design a circuit that performs both addition and subtraction based on a mode signal M :

- If $M = 0$, perform $A + B$.
- If $M = 1$, perform $A - B$.

Implement the design using XOR gates and Full Adders.

- **Maximum of Two Numbers**

Design a circuit that outputs the larger of two n -bit numbers:

$$\max(A, B)$$

Use a subtractor to compare the numbers and select the larger value as the output.

- **Minimum of Two Numbers**

Design a circuit that outputs the smaller of two n -bit numbers:

$$\min(A, B)$$

Use a subtractor to compare the numbers and select the smaller value as the output.